Empirical CDF & PDF

Estimating Distribution from Data

Kiara (Jihyun) Kim

Last updated: September 10, 2026

Econ 4320

Today's Plan

1. **Empirical CDF:** the simplest way to estimate a distribution from data
2. **Empirical PDF:** use Kernel Density Estimation (KDE) to draw a smooth curve that follows your data

Why this matters for us

The first step of empirical auction is to estimate the bid distribution from data. Today you will learn how that estimation works, how to prompt AI to do it correctly and how to test it.

Example Bid Data

Bidder	Placed	Amount
Bidder #27	**11/7/2017 7:26:01 AM**	**$826**
Bidder #30	11/7/2017 9:58:54 AM	$825
Bidder #27	11/7/2017 7:26:01 AM	$797
Bidder #30	11/7/2017 9:58:19 AM	$796
Bidder #27	11/7/2017 7:26:01 AM	$765
Bidder #30	11/6/2017 4:54:29 PM	$764
Bidder #27	11/6/2017 3:58 PM	$751
Bidder #30	11/6/2017 4:54:10 PM	$662
Bidder #27	11/6/2017 3:58	$561

Preview: Where We Are Headed

What we observe: bidders' bids ($b \sim G$)

What we want: bidders' values ($v \sim F$) $\rightarrow$ How will bidders bid under a different selling rule?

Since we don't know G and g, we estimate them as $\hat{G}$ and $\hat{g}$.

Plugging them in gives the value estimate:

$$\hat{v} = b + \frac{\hat{G}(b)}{(n-1)\,\hat{g}(b)}$$

Today's lecture is about obtaining the estimates $\hat{G}(\cdot)$ and $\hat{g}(\cdot)$ from observed bids.

What Are CDF and PDF

Suppose bid values are spread somewhere between \$0 and \$10.

CDF: $F(x)$

- $F(x) = \Pr(\text{bid} \leq x)$
- Sum of $f(\cdot)$ up to the point x
- What fraction of bids are at or below x
- Always goes from 0 to 1

PDF: $f(x)$

- $f(x) = \Pr(\text{bid} = x)$; **height** of the curve at x
- High $\Rightarrow$ many bids near that value
- Area under the whole curve $= 1$
- Derivative of $F(x)$

PDF $\rightarrow$ CDF: integrate. CDF $\rightarrow$ PDF: differentiate.
In practice, we **estimate them separately** (more on this later on)

The Problem: We Never Know the True Distribution

In the real world, bidders' values come from some unknown distribution F.
We only observe a **finite sample of bids**.

Our job

Use the observed bids to estimate F and f as accurately as possible.

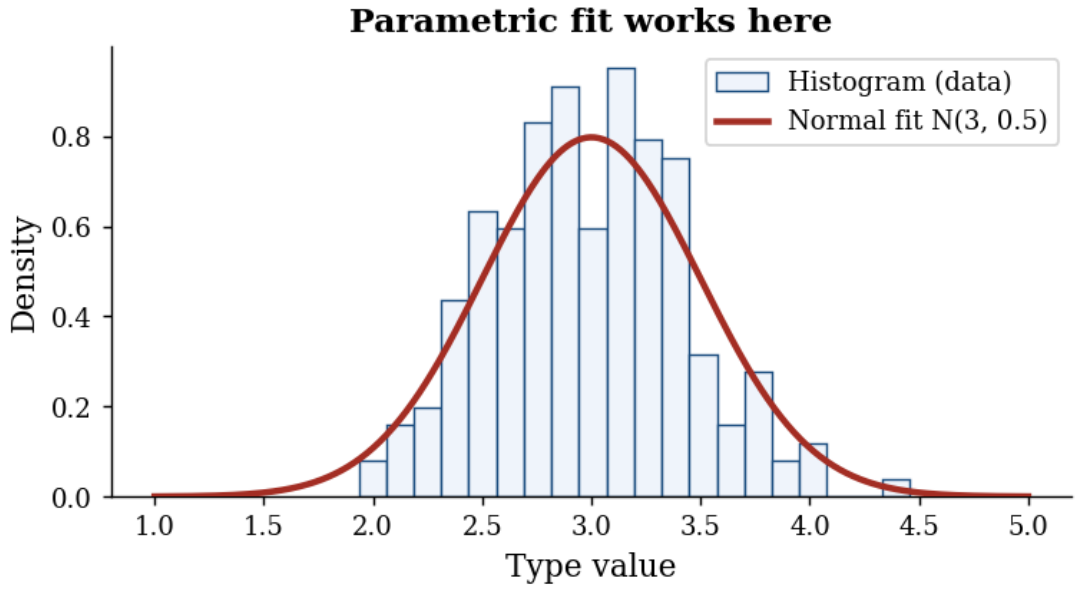
Two approaches:

1. **Parametric:** Assume a specific shape (e.g., Normal distribution), then fit the parameters.
2. **Nonparametric:** Let the data tell us the shape (no assumptions).

Approach 1: Assume a Shape (Parametric)

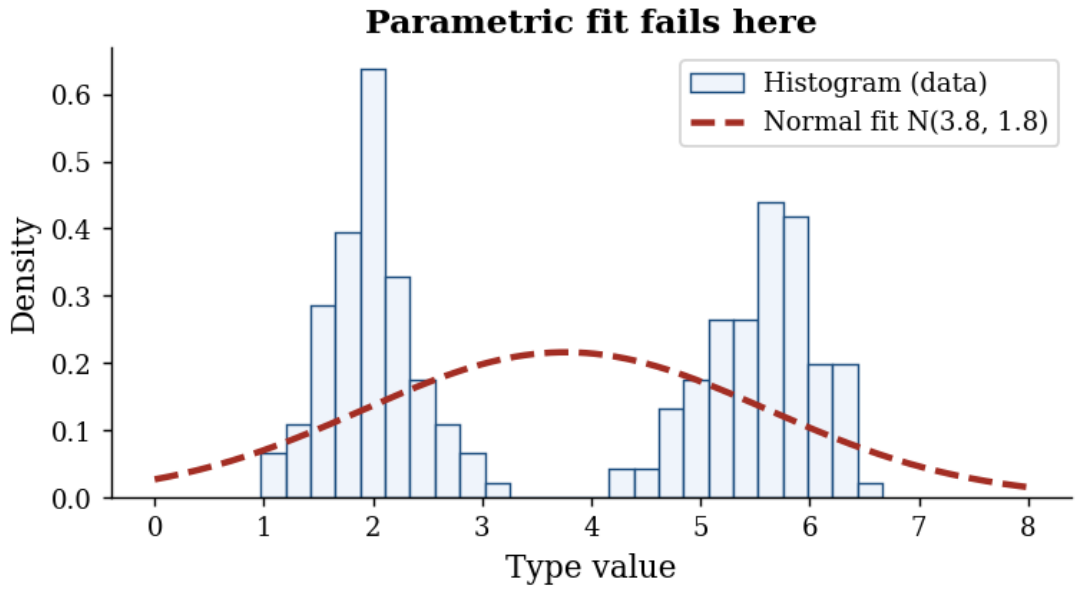
The idea

Pick a well-known distribution family (Normal, Log-Normal, . . .) and fit it to your data.



Works great when the true distribution *is* that shape. Simple to explain and code.

When Parametric Fails

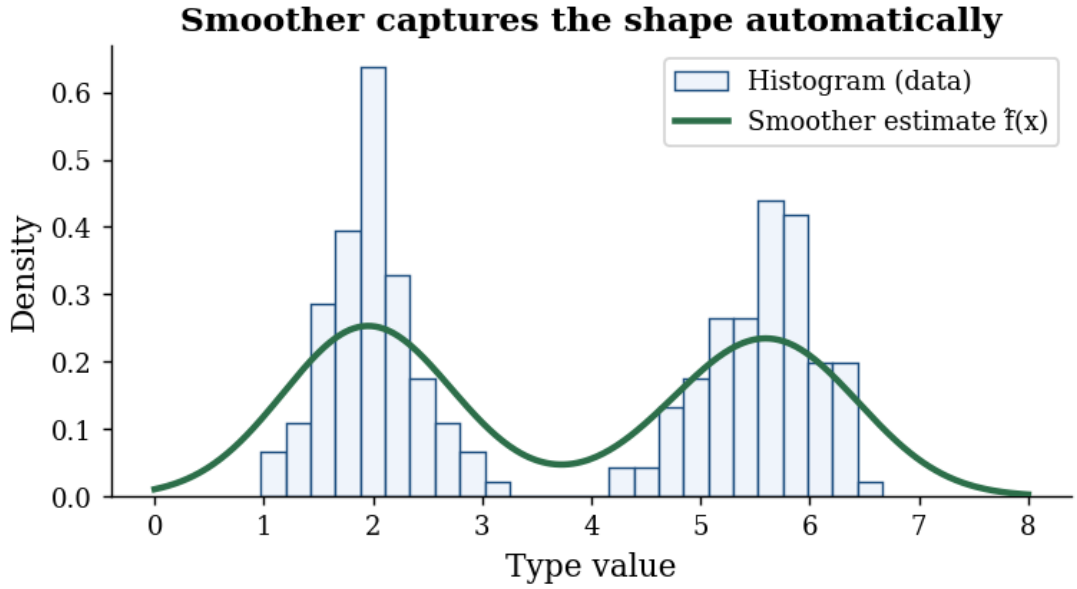


Two groups of bidders with very different values produce a **two-humped** histogram.

A single Normal distribution fits neither hump well.

We need a more flexible approach that lets the data tell us the shape.

Approach 2: Let the Data Draw the Curve



Same two-humped data. The **smoother** automatically finds both peaks without us telling it anything about the shape.

Wherever the data are **dense**, the curve is **high**.
Wherever the data are **sparse**, the curve is **low**.

Estimating the Empirical CDF

Let T be the total number of observations.

The simplest possible estimator of F

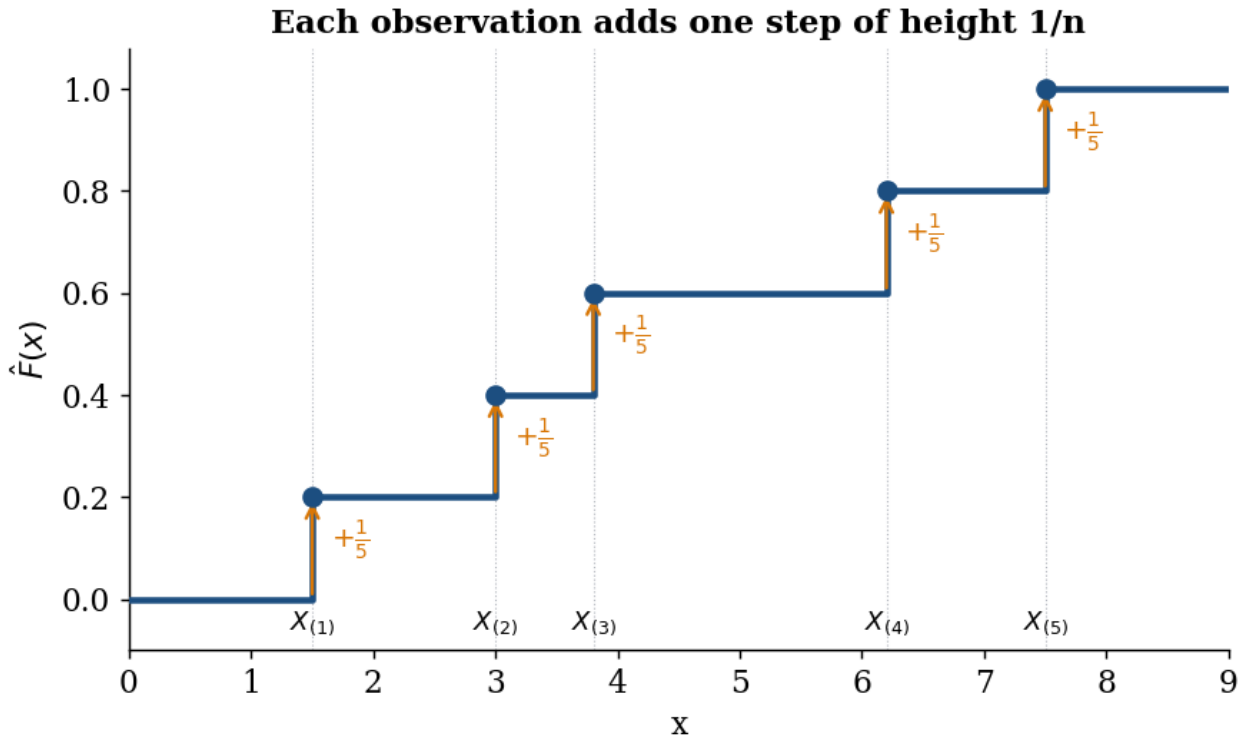
$$\hat{F}(x) = \frac{\text{number of observations} \leq x}{T} = \frac{1}{T} \sum_{i=1}^{T} \mathbf{1}(X_i \leq x)$$

- Count how many bids are at or below x, divide by sample size T
- Every time x passes a data point, $\hat{F}$ jumps up by $\frac{1}{T}$

Example

If you observed five bids ($T = 5$) and three of them are below \$4, then $\hat{F}(4) = Pr(b \leq 4) = 3/5 = 0.6$.

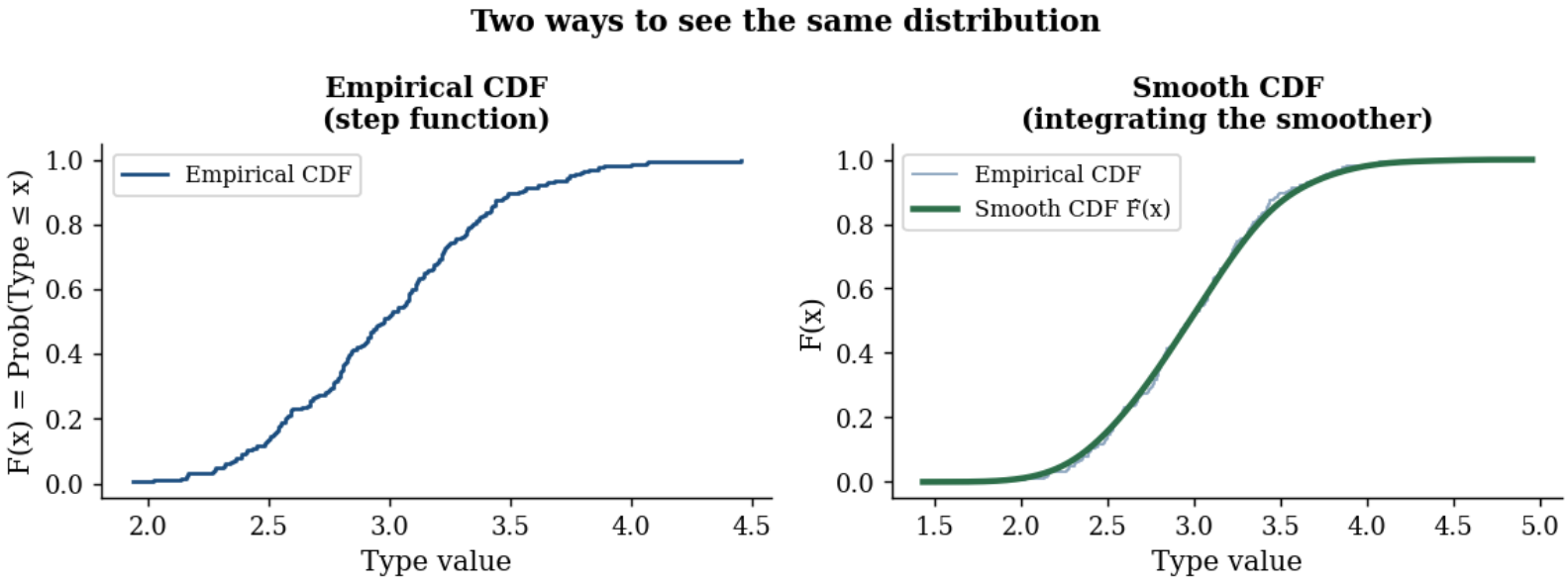
Empirical CDF: What It Looks Like



With $T = 5$ observations, each data point adds a step of height $1/5 = 0.2$.

After all 5 steps, $\hat{F}$ reaches 1.

Empirical CDF on Real Data



Left: The empirical CDF; has step-up pattern

Right: A *smooth* CDF obtained by integrating the KDE smoother instead.

Why We Also Need a Smooth PDF

The GPV formula needs $g(b)$, not just $G(b)$

Recall: $\hat{v} = b + \dfrac{\hat{G}(b)}{(n-1)\,\hat{g}(b)}$

- We need $\hat{g}(b)$, the **PDF** of bids at point b
- We cannot differentiate a step function to get a usable PDF:

  The empirical CDF $\hat{G}(b)$ is a staircase with flat steps, so its derivative is zero almost everywhere, with spikes at each data point

Solution

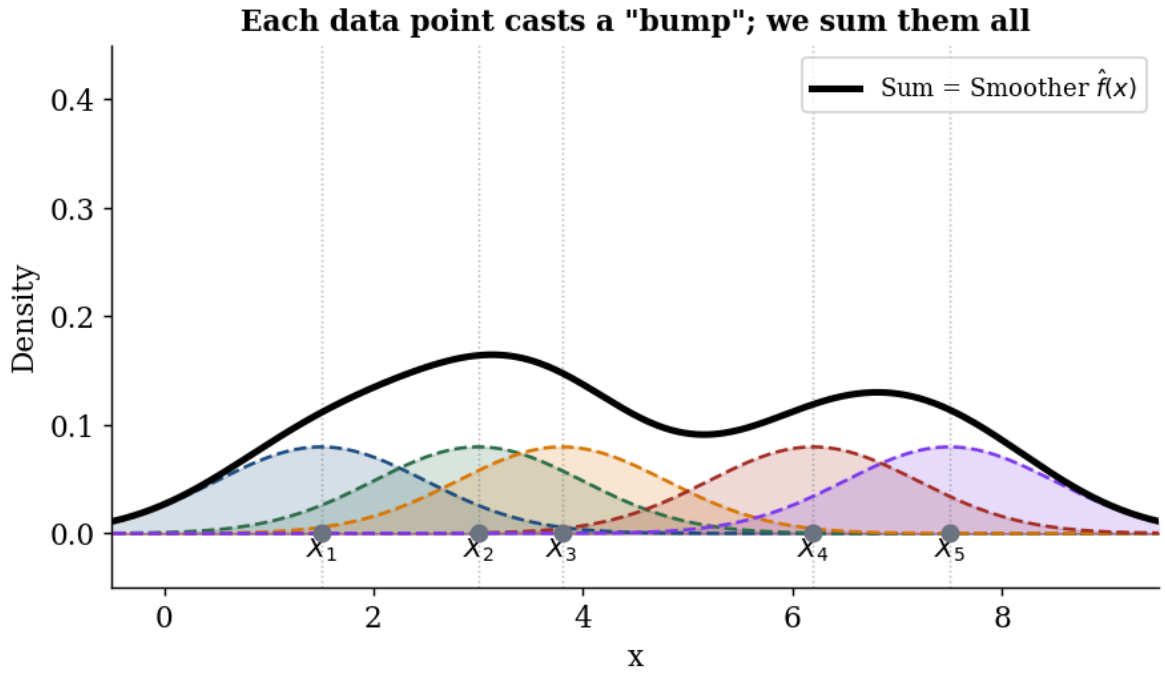
Use a **smoother** (kernel density estimator) to directly estimate the PDF in a smooth, differentiable way.

Estimating the PDF: The Idea Behind the Smoother

The idea:

Place a small **bell-shaped bump** on top of each observation.

Sum all the bumps together $\rightarrow$ the total is your estimated PDF.



Each data point X_i casts one bump centered at that point.

Where data are dense, bumps pile up $\Rightarrow$ the curve is high.

Where data are sparse, few bumps $\Rightarrow$ the curve is low.

The Formula: Kernel Density Estimator

Kernel Density Estimator (KDE)

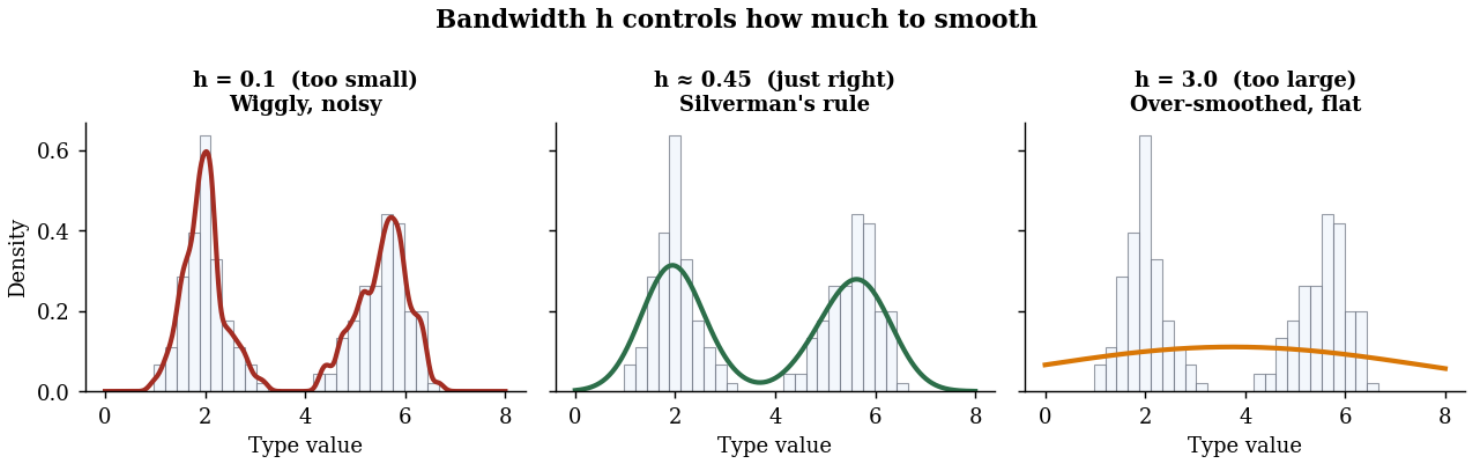
$$\hat{g}_h(x) = \frac{1}{Th} \sum_{i=1}^{T} K\left(\frac{x - X_i}{h}\right)$$

- T = number of observations (To avoid notational confusion with number of bidders, I use T. However, n is also common notation.)
- $K(\cdot)$ = the **bump shape** ("kernel") — usually a bell curve
- h = **bandwidth**: controls how *wide* each bump is
- At each x: add up contributions from nearby points, weighted by closeness

Most common kernel

Gaussian: $K(u) = \frac{1}{\sqrt{2\pi}} e^{-u^2/2}$ (a standard bell curve)

The Bandwidth h: The Smoothness Knob



- h **too small** — every tiny fluctuation in the data shows up; very noisy
- h **just right** — smooth curve that still follows the real features
- h **too large** — everything gets blurred; real peaks disappear

Choosing h: Silverman's Rule of Thumb

Silverman's rule (the standard default)

$$h = 1.06 \times \hat{\sigma} \times T^{-1/5}$$

where $\hat{\sigma}$ = sample standard deviation of the data.

- As T grows, h shrinks — more data means you can zoom in more
- **Python** (`scipy.stats.gaussian_kde`) and **R** (`density(x)`) uses Silverman by default

Practical note

Silverman's rule works well when the data look like a bell curve. For strongly two-humped data, it may over-smooth ⇒ **Plot the estimated PDF against the histogram**

Testing AI-Generated Code: Test Against Known Truth

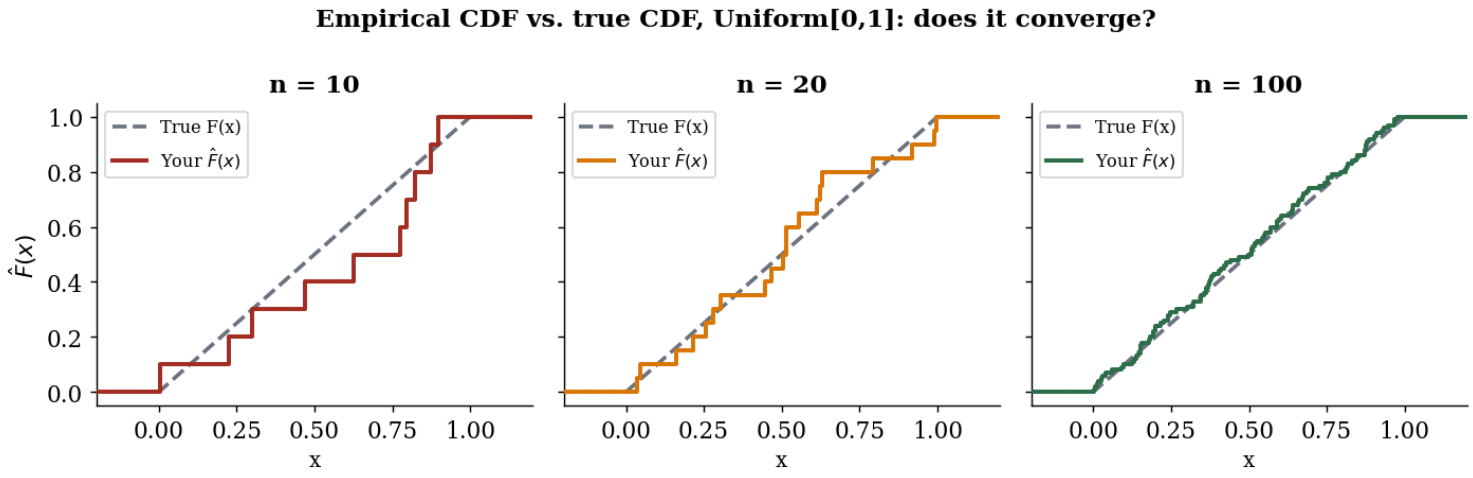
The recipe:

1. Draw n observations from a **known** distribution (Uniform, Normal or Bimodal)
2. Run your (or the AI's) estimator on this sample
3. Overlay the estimate against the **true** G and g
4. Repeat for larger n (e.g. 20, 100, 1000): a correct estimator should **converge** to the truth

   (*Note:* Here, I use n instead of T to refer to sample size instead of number of bidders)

A wrong formula will **not** converge to the truth as n grows. Convergence is your evidence that the code is correct.

Example of Convergence



Draw n points from Uniform$[0, 1]$ whose true CDF is $G(x) = x$ (the diagonal line).

Compare $n = 10$ vs $n = 100$:

If your code is right, more data always means a better fit.

Summary

Concept	**Key takeaway**
CDF $G(x)$	Fraction of bids at or below x; ranges from 0 to 1
PDF $g(x)$	Height = how likely that bid value is; area = 1
Empirical CDF	Just count: $\hat{F}(x) = \frac{\#\{X_i \leq x\}}{n}$
KDE	Place a bump on each data point, sum them up
Bandwidth h	Width of each bump; Silverman's rule is a safe default
Why KDE (not histogram)	Smoother, more accurate, needed for empirical auction
Testing your code	Always validate against a known distribution

Appendix: Estimating the PDF is Harder

CDF: fast convergence

The empirical CDF $\hat{F}(x)$ converges to the true $F(x)$ at rate $n^{-1/2}$

PDF: slower convergence

The KDE $\hat{f}(x)$ converges at rate $n^{-2/5} \rightarrow$ slower, even with the best possible bandwidth.

Intuition

Estimating $f(x)$ is like estimating the *slope* of $F(x)$.

- Slopes are sensitive: a small error in F near x gets amplified into a big error in $f(x) = F'(x)$.
- Smoothing controls this, but at a cost in how fast the estimator converges.

This is why we estimate f and F with separate dedicated methods, each tuned for that task.

Appendix: Bias–Variance Trade-off in h

Mean Squared Error of $\hat{f}(x)$

$$\mathrm{MSE}[\hat{f}(x)] \approx \underbrace{h^4 \cdot b(x)^2}_{\text{bias}^2} + \underbrace{\frac{1}{nh} \cdot v(x)}_{\text{variance}}$$

- **Bias** grows with h^4: a large h gives a smooth but systematically wrong curve
- **Variance** shrinks with nh: a small h gives noisy but locally accurate estimates
- Silverman's rule minimises this trade-off by plugging in a Normal approximation for $b(x)$

$\Rightarrow$ There is no perfect h!